Light

Computational Thinking

Over **1,100,000+ marks** of exam-aligned content across **67 qualifications**. 4,200+ auto-marked assessments & long-answer practice. No login required.

Assessment Hub | Notes / Textbook | Long-Answer Practice

Assessment Hub
Long-Answer
Notes Index
What you must know
Toolkit
1. Decomposition and abstraction
2. Algorithms
3. Standard algorithms and truth tables
Worked examples
Quick reference
Typical pitfalls

Edexcel GCSE Computer Science — **Computational Thinking (1CP2/01 Topic 1)**. This topic covers how computer scientists approach and solve problems, including decomposition, abstraction, designing algorithms, using trace tables, understanding error types, and working with standard sorting and searching algorithms.
Exam questions typically ask you to **identify** programming constructs, **complete** trace tables, **write** algorithms, and **construct** truth tables. The 2024 Paper 1 included 16 marks on computational thinking, covering decomposition, subprogram benefits, loop identification, error types, operators, and algorithm efficiency.

Why computational thinking matters

Computational thinking is the foundation of computer science. It provides the mental tools to:

- **Break down complex problems:** Decomposition makes large problems manageable
- **Focus on what matters:** Abstraction removes unnecessary detail
- **Design solutions:** Algorithms provide step-by-step instructions
- **Find and fix errors:** Trace tables and testing reveal bugs

- **Choose efficient methods:** Standard algorithms solve common problems effectively
- **Exam relevance:** Topic 1 is worth approximately 16 marks on Paper 1

What you must know

This checklist covers the key exam-visible concepts from Edexcel 1CP2 Topic 1.

- **Define** decomposition and abstraction and explain their benefits
- **State** two or more benefits of using subprograms
- **Follow and write** algorithms using flowcharts, pseudocode, and program code
- **Identify** sequence, selection, repetition (count-controlled, condition-controlled), and iteration
- **Use** variables, constants, and data structures (strings, arrays, records)
- **Apply** arithmetic, relational, and logical operators
- **Complete** trace tables to determine variable values
- **Define** syntax errors, logic errors, and runtime errors
- **Explain** how bubble sort, merge sort, linear search, and binary search work
- **Construct** truth tables with up to three inputs using AND, OR, NOT

> **Specification link:** This topic covers Edexcel 1CP2 Paper 1 Topic 1: Computational thinking (1.1 Decomposition and abstraction, 1.2 Algorithms, 1.3 Truth tables).

Toolkit

These are the key definitions, operators, and patterns you need for computational thinking questions.

Key definitions

Decomposition

Breaking a problem or solution down into smaller, more manageable parts.
Exam phrase: "Breaking down into smaller parts."

Abstraction

Hiding or removing unnecessary detail to focus on what is important.
Exam phrase: "Removing/hiding unnecessary detail."

Programming constructs

Construct	Description	Example keyword/pattern
Sequence	Instructions executed one after another in order	Lines 1, 2, 3...
Selection	Decision — different paths based on a condition	if, elif, else

Construct	Description	Example keyword/pattern
Count-controlled repetition	Loop that runs a fixed number of times	for i in range()
Condition-controlled repetition	Loop that runs while a condition is true	while condition:
Iteration over data structure	Loop through every item in a list/array	for item in list:

Operators

Arithmetic operators

- **+** Addition
- **−** Subtraction
- * Multiplication
- **/** Division
- **//** Integer division
- **%** Modulus (remainder)
- ** Exponentiation

Relational operators

- **==** Equal to
- **!=** Not equal to
- **<** Less than
- **>** Greater than
- **<=** Less than or equal
- **>=** Greater than or equal

Logical operators

- **AND** Both must be true
- **OR** At least one must be true
- **NOT** Inverts true/false

Relational tests return **True** or **False**.

Error types

Error type	When it occurs	Example
Syntax error	Code breaks the rules/grammar of the programming language	Missing colon, misspelled keyword, unmatched brackets
Logic error	Code runs but produces incorrect results	Using < instead of >, wrong formula
Runtime error	Code crashes during execution due to impossible operation	Division by zero, accessing invalid array index

Flowchart symbols

```
┌─────────────┐ │ TERMINATOR │ Oval/Rounded rectangle — Start or End
└─────────────┘ ┌─────────────┐ │ PROCESS │ Rectangle — Instruction or
calculation └─────────────┘ ┌─────────────┐ / \ │ INPUT/OUTPUT │
Parallelogram — Input or output \ / └─────────────┘ /\ / \ / \ │ YES/ │
Diamond — Decision (Yes/No or True/False) │ NO │ \ / \ / \/
┌─────────────┐ │ SUBPROGRAM │ Rectangle with vertical lines — Call
subprogram │— —│ └─────────────┘ │ ▼ Arrow — Shows flow of control
```

1. Decomposition and abstraction

Decomposition and abstraction are two fundamental techniques that computer scientists use to tackle complex problems. They work together to make problems easier to understand and solve.

1.1 Decomposition

Decomposition is breaking a problem or solution down into smaller, more manageable parts. Each part can then be solved independently before being combined into a complete solution.

Benefits of decomposition

- Makes complex problems easier to understand
- Smaller parts are easier to solve
- Different people can work on different parts (teamwork)
- Each part can be tested independently
- Parts can be reused in other projects
- Easier to identify where errors occur

How to decompose a problem

1. **Identify the main task:** What is the overall goal?
2. **Break it into major components:** What are the big parts?
3. **Break components into smaller tasks:** Keep breaking down until tasks are simple
4. **Identify connections:** How do the parts link together?

Example: Making a game

A game can be decomposed into:

- User input handling
- Game logic (rules, scoring)
- Graphics display
- Sound effects
- Saving/loading game state

Each component is simpler to build than the whole game. A programmer can focus on one area at a time.

1.2 Abstraction

Abstraction is hiding or removing unnecessary detail to focus on what is important for the current task. It simplifies complex systems by showing only the relevant information.

Benefits of abstraction

- Reduces complexity by hiding detail
- Allows focus on the essential features
- Makes problems easier to model
- Enables creation of general solutions
- Helps manage large amounts of information

Types of abstraction

- **Data abstraction:** Hiding how data is stored (e.g., using a list without knowing how memory works)
- **Procedural abstraction:** Using a subprogram without knowing how it works inside
- **Representational abstraction:** Using simplified models (e.g., maps, diagrams)

Example: London Tube map

The Tube map is an abstraction of London's underground railway:

- **Included:** Stations, lines, connections
- **Hidden:** Exact distances, geographical accuracy, tunnels, depth underground

The map focuses on what passengers need (how to get from A to B) and removes what they don't (the engineering details).

Abstraction in programming

When you use `print("Hello")`, you don't need to know how the computer sends signals to the screen. The details are hidden — you just use the function.

> **Exam tip:** Don't confuse decomposition and abstraction. Decomposition = breaking into parts. Abstraction = hiding detail. They are related but different concepts.

1.3 Subprograms

Subprograms (also called procedures, functions, or subroutines) are self-contained blocks of code that perform a specific task. They are a key result of decomposition and enable abstraction by hiding implementation details.

Types of subprograms

- **Procedure:** Performs a task but does not return a value
- **Function:** Performs a task and returns a value

Benefits of subprograms

Benefit	Explanation
Code only written once	Saves development time — no need to repeat the same code
Can be called many times	Reusability — use the same code in multiple places
Only debugged once	Fix errors in one place, not multiple copies
Easier to find errors	Can test each subprogram independently
Hides implementation details	Abstraction — programmers can use it without knowing how it works
Can be grouped into libraries	Shared with other programmers and projects
Makes programs easier to read	Clear structure with named blocks of code
Enables teamwork	Different programmers can work on different subprograms

Exam tip: The 2024 exam asked for two benefits of subprograms. Common answers include: "Code only written once", "Can be called many times", "Only debugged once", "Makes programs easier to read", "Enables teamwork", and "Hides implementation details."

Worked Example 1 — Defining decomposition

Question: Identify the term that means breaking a problem or solution down into smaller parts. [1 mark]

Model answer:
Decomposition **(1)**
Note: This was a multiple choice question in 2024. The distractors were Abstraction, Computation, and Evaluation.

Exam tip: Decomposition = breaking DOWN. Abstraction = hiding/removing detail.

Worked Example 2 — Benefits of subprograms

Question: State two benefits of subprograms. [2 marks]

Model answer:
The subprogram code only has to be written once / saves development time **(1)**.
The subprogram can be called many times (reusability) **(1)**.
Also acceptable: Only debugged once / Easier to find errors / Hides details (abstraction) / Can be shared in libraries / Makes programs easier to read / Enables teamwork.

Exam tip: Don't just say "easier" or "faster" — explain WHY it's easier or faster.

Worked Example 3 — Explaining abstraction

Question: Explain what is meant by abstraction and give an example. [3 marks]

Model answer:
Abstraction is hiding or removing unnecessary detail **(1)**.
This allows focus on what is important for the task **(1)**.
Example: A road map shows roads and towns but hides details like buildings and trees **(1)**.

Exam tip: Good examples of abstraction include maps, icons, models, and simplified diagrams.

Worked Example 4 — Decomposition example

Question: A school wants to create a system to manage library books. Describe how decomposition could be used. [2 marks]

Model answer:
Break the library system down into smaller parts **(1)**.
For example: borrowing books, returning books, searching for books, adding new books, managing user accounts **(1)**.

Exam tip: When asked to apply decomposition, list 3-4 specific parts the system could be broken into.

2. Algorithms

An **algorithm** is a step-by-step set of instructions designed to solve a problem or complete a task. Algorithms must be:

- **Precise:** Each step must be clearly defined
- **Unambiguous:** There should be only one way to interpret each step
- **Finite:** The algorithm must eventually stop
- **Effective:** Each step must be doable

Algorithms can be represented in different ways: flowcharts, pseudocode, or program code. Each has advantages for different purposes.

2.1 Representing algorithms

Flowcharts

Visual representations using standard symbols (see Toolkit). Flowcharts are useful for:

- Showing the flow of control through an algorithm
- Visualising decision points
- Communicating algorithms to non-programmers
- Planning before writing code

Pseudocode

Informal written description using English-like statements. Pseudocode is not a real programming language — no strict syntax is required. This makes it:

- Easy to read and understand
- Quick to write
- Good for planning
- Language-independent

```
INPUT name IF name == "Admin" THEN OUTPUT "Welcome, Administrator" ELSE
OUTPUT "Hello, " + name ENDIF
```

> **Exam tip:** Edexcel uses its own pseudocode style in exams. Always follow the syntax given in the question paper — the examples here show the general structure.

Program code

Actual code in a programming language like Python. This is what the computer executes:

```
name = input("Enter name: ") if name == "Admin": print("Welcome, Administrator") else: print("Hello, " + name)
```

2.2 Programming constructs

The 2024 exam asked students to identify different constructs in given code. Understanding the difference between these is essential.

Sequence

Lines executed one after another, in order from top to bottom. This is the default — every program uses sequence.

```
name = input("Enter name: ") print("Hello " + name) print("Welcome to the program")
```

Selection (if/else)

Choosing between different paths based on a condition. Only one branch is executed.

```
if colour == "Green": print("Green is my favourite") elif colour == "Blue": print("Blue is nice too") else: print(colour + " is a good colour")
```

Key words: if, elif (else if), else

Condition-controlled loop (while)

Repeats while a condition is true. May run zero or more times. The condition is checked before each iteration.

```
password = "" while password != "secret": password = input("Enter password: ") print("Access granted")
```

Key word: while

Count-controlled loop (for...range)

Repeats a fixed number of times. You know in advance how many iterations there will be.

```
for i in range(5): print(i) # Prints 0, 1, 2, 3, 4
```

Key words: for, in, range()

Iteration over data structure (for...in)

Loops through every item in a list, array, or string.

```
colours = ["Red", "Green", "Blue"] for item in colours: print(item)
```

Key words: for, in, followed by a list/array name (not range)

Important: Don't confuse `for i in range(5)` (count-controlled) with `for item in myList` (iteration over structure). The first uses range(), the second uses a data structure name.

2.3 Variables, constants and data structures

Variables

Named storage locations whose values can change during program execution.

```
score = 0 score = score + 10
```

Constants

Named values that do not change. Conventionally written in UPPERCASE.

```
MAX_SCORE = 100 PI = 3.14159
```

Data structures

Structure	Description	Example
String	Sequence of characters	"Hello World"
Array (1D)	Ordered collection of items of the same type	["Red", "Green", "Blue"]
Array (2D)	Grid/table of items (rows and columns)	[[1,2,3], [4,5,6], [7,8,9]]
Record	Collection of related fields (can be different types)	student.surname = "Smith" student.age = 25 student.hasPaid = True

2.4 Operators in detail

Integer division (//) and modulus (%)

These are commonly tested operators that students often confuse:

Expression	Result	Explanation
5 / 2	2.5	Normal division
5 // 2	2	Integer division — whole number only
5 % 2	1	Modulus — remainder after division

Expression	Result	Explanation
17 // 5	3	17 ÷ 5 = 3 remainder 2, so // gives 3
17 % 5	2	17 ÷ 5 = 3 remainder 2, so % gives 2

> **Exam tip:** The 2024 exam asked what 5 // 2 equals. The answer is 2 (integer division gives the whole number part only).

Common uses of modulus (%)

The modulus operator is particularly useful for:

- **Checking if even/odd:** `if number % 2 == 0` means even
- **Wrapping around:** `position % 10` keeps values 0-9
- **Extracting digits:** `number % 10` gets the last digit

2.5 Trace tables

A **trace table** is a technique for following the execution of an algorithm step by step. Each row shows the values of all variables after each instruction is executed. Trace tables help you:

- Determine the output of an algorithm for given input
- Find the value of a variable at any specific point
- Identify logic errors by seeing where values go wrong
- Understand how loops and conditions affect program flow

How to complete a trace table

1. Create columns for each variable, plus columns for conditions and output
2. Start with initial values (or leave blank if not yet assigned)
3. Work through each line of code in order
4. When a variable changes, write the new value in that row
5. For conditions, write True or False
6. Only fill in columns that change on each line

Example trace table

For this algorithm:

```
x = 1 y = 0 while x <= 3: y = y + x x = x + 1 print(y)
```

Step	x	y	x <= 3	Output
1	1	0	—	—
2	1	1	True	—
3	2	1	—	—
4	2	3	True	—
5	3	3	—	—
6	3	6	True	—

Step	x	y	x <= 3	Output
7	4	6	False	6

Notice how the loop continues while x <= 3. When x becomes 4, the condition is False and the loop exits. The algorithm calculates 1 + 2 + 3 = 6.

Common trace table patterns

Pattern	What to look for
Counter	Variable increases by 1 each iteration (i = i + 1)
Accumulator	Variable accumulates values (total = total + value)
Flag	Boolean that changes when condition met (found = True)
Max/Min	Variable updates when larger/smaller value found

Exam tip: In trace tables, work methodically line by line. Don't try to jump ahead or predict — follow the code exactly as the computer would.

2.6 Error types

Syntax error

Code breaks the rules of the programming language.

Detected: Before/during translation

Examples:

- Missing colon after if
- Misspelled keyword
- Unmatched brackets

Logic error

Code runs but produces wrong results.

Detected: By testing/user

Examples:

- Using < instead of >
- Wrong formula
- Off-by-one errors

Runtime error

Code crashes during execution.

Detected: When program runs

Examples:

- Division by zero
- Invalid array index
- File not found

Worked Example 5 — Identifying constructs in code

Question: Look at this algorithm:

```
1 theColours = ["Green", "Blue", "Yellow", "Red"] 2 colour = "" 3 4
for item in theColours: 5 print(item) 6 7 colour = input("Enter a
colour: ") 8 while colour != "": 9 if colour == "Green": 10
print("Green is my favourite colour") 11 else: 12 print(colour + " is
a good colour") 13 colour = input("Enter a colour: ")
```

(i) Give the first line number of a condition-controlled loop. [1 mark]
(ii) Give the first line number of iteration over every item in a data structure. [1 mark]
(iii) Give the line numbers of a selection. [1 mark]

Model answer:
(i) Line 8 (the while loop) **(1)**
(ii) Line 4 (for item in theColours) **(1)**
(iii) Lines 9—12 (the if/else block) **(1)**

Exam tip: This was in the 2024 exam. Know the difference between while (condition-controlled) and for...in (iteration over data structure).

Worked Example 6 — Defining syntax error

Question: Define the term 'syntax error'. [1 mark]

Model answer:
Code that breaks/violates the rules/grammar of the programming language **(1)**.
Note: Do not just give an example — you must give a definition.

Exam tip: For "define" questions, give a general definition, not just an example.

Worked Example 7 — Explaining runtime error

Question: Runtime errors happen when a program is executing. Explain a runtime error. [2 marks]

Model answer:
The program crashes/stops **(1)**.
Because the operation the computer is asked to do is impossible (e.g., division by zero, accessing invalid array index) **(1)**.

Exam tip: Runtime errors cause the program to crash. They happen during execution, not during translation.

Worked Example 8 — Operator result

Question: Identify the result of 5 // 2. [1 mark]

Model answer:
2 **(1)**
// is integer division — it gives only the whole number part.
5 ÷ 2 = 2.5, but // removes the decimal, leaving 2.

Exam tip: Don't confuse // (integer division = 2) with % (modulus/remainder = 1) or / (division = 2.5).

3. Standard algorithms and truth tables

Standard algorithms are well-known solutions to common problems. You need to understand how they work and be able to compare their characteristics.

3.1 Searching algorithms

Linear search

Checks each item in the list one by one from the start until the target is found or the end is reached.

- **Works on:** Any list (sorted or unsorted)
- **Method:** Check each item in turn
- **Efficiency:** Slower for large lists
- **Best case:** Target is first item (1 comparison)
- **Worst case:** Target is last or not present (n comparisons)

Linear search pseudocode

```
SET found = False SET index = 0 WHILE index < length of list AND found =
False IF list[index] = target THEN SET found = True ELSE SET index = index
+ 1 ENDIF ENDWHILE IF found = True THEN OUTPUT "Found at position " +
index ELSE OUTPUT "Not found" ENDIF
```

```
List: [5, 2, 8, 1, 9] Find: 8 Step 1: Check 5 — not 8 Step 2: Check 2 — not
8 Step 3: Check 8 — FOUND at position 2
```

Binary search

Repeatedly divides a sorted list in half, eliminating half the remaining items each time.

- **Works on:** Sorted lists only
- **Method:** Divide and conquer

- **Efficiency:** Much faster for large lists
- **Best case:** Target is middle item (1 comparison)
- **Worst case:** Many halvings needed (far fewer than linear search)

Binary search pseudocode

```
SET found = False SET low = 0 SET high = length of list - 1 WHILE low <=
high AND found = False SET mid = (low + high) // 2 IF list[mid] = target
THEN SET found = True ELSEIF list[mid] < target THEN SET low = mid + 1
ELSE SET high = mid - 1 ENDIF ENDWHILE
```

```
List: [1, 2, 5, 8, 9] Find: 8 Step 1: Middle is 5. 8 > 5, search right half
Step 2: Middle is 8. FOUND!
```

Exam tip: Binary search is a "divide and conquer" algorithm. This was asked in the 2024 exam.

3.2 Sorting algorithms

Bubble sort

Repeatedly compares adjacent pairs and swaps them if in wrong order. After each pass, the largest unsorted item "bubbles" to its correct position at the end.

- **Method:** Compare adjacent pairs, swap if needed
- **Passes:** Multiple passes through the list
- **Efficiency:** Simple but slow for large lists
- **Not divide and conquer**

Bubble sort pseudocode

```
SET swapped = True WHILE swapped = True SET swapped = False FOR i = 0 TO
length of list - 2 IF list[i] > list[i + 1] THEN SWAP list[i] AND list[i +
1] SET swapped = True ENDIF NEXT i ENDWHILE
```

```
Pass 1: [5,2,8,1] → [2,5,8,1] → [2,5,8,1] → [2,5,1,8] Pass 2: [2,5,1,8] →
[2,5,1,8] → [2,1,5,8] Pass 3: [2,1,5,8] → [1,2,5,8] Result: [1,2,5,8]
```

Notice how after each pass, the largest unsorted item moves to its correct position at the end. The algorithm stops when a complete pass makes no swaps.

Merge sort

Divides the list into halves repeatedly until single items remain, then merges them back together in sorted order.

- **Method:** Divide into halves, merge in order
- **Efficiency:** Faster for large lists
- **Uses divide and conquer**
- **Extra memory:** Needs additional storage for merging

How merge sort works

1. **Divide:** Split the list into two halves

2. **Recurse:** Keep splitting until you have single items
3. **Merge:** Combine pairs back together in sorted order

```
[5, 2, 8, 1] │ ▼ Split ┌──────┴──────┐ [5, 2] [8, 1] │ │ ▼ Split ▼ Split
┌──┴──┐ ┌──┴──┐ [5] [2] [8] [1] │ │ │ │ └──┬──┘ └──┬──┘ ▼ Merge ▼ Merge [2,
5] [1, 8] │ │ └──────┬──────┘ ▼ Merge [1, 2, 5, 8]
```

When merging, compare the first items of each half. The smaller one goes into the result first. Repeat until both halves are empty.

3.3 Algorithm efficiency

Efficiency measures how well an algorithm performs. The 2024 exam asked about the effect of comparisons on execution time.

Factor	Effect on efficiency
Number of comparisons	More comparisons = slower execution
Number of passes through data	More passes = slower execution
Memory usage	More memory = may be slower, limits size of data

Comparing search algorithms

Aspect	Linear search	Binary search
Requires sorted data?	No	Yes
Divide and conquer?	No	Yes
Speed on small lists	Fast	Fast
Speed on large lists	Slow	Fast
Comparisons (worst case)	All items checked	Far fewer (halves each time)

Comparing sort algorithms

Aspect	Bubble sort	Merge sort
Divide and conquer?	No	Yes
Extra memory needed?	No (in-place)	Yes
Speed on small lists	Adequate	Good
Speed on large lists	Very slow	Fast
Simplicity	Very simple	More complex

Key comparison:

- **Binary search** is a divide and conquer algorithm
- **Bubble sort** is NOT a divide and conquer algorithm
- **Merge sort** IS a divide and conquer algorithm

3.4 Truth tables

Truth tables show all possible combinations of inputs and the resulting output for logical expressions. You need to work with AND, OR, and NOT with up to three inputs.

Understanding logical operators

- **AND:** Returns True only if BOTH inputs are True. Think: "both must be true".
- **OR:** Returns True if AT LEAST ONE input is True. Think: "one or both must be true".
- **NOT:** Inverts the value. True becomes False, False becomes True.

Basic truth tables

AND

Both inputs must be True

A	B	A AND B
0	0	0
0	1	0
1	0	0
1	1	1

Only the last row gives 1.

OR

At least one input must be True

A	B	A OR B
0	0	0
0	1	1
1	0	1
1	1	1

Only the first row gives 0.

NOT

Inverts the input

A	NOT A
0	1
1	0

Flips 0 to 1 and 1 to 0.

How to construct a truth table

1. **Count inputs:** For n inputs, you need 2^n rows (2 inputs = 4 rows, 3 inputs = 8 rows)
2. **List all combinations:** Start with all 0s, count up in binary
3. **Add intermediate columns:** Break complex expressions into steps

4. **Apply operators in order:** Work from inside brackets outward

Three-input truth table example

Expression: (A AND B) OR C

Step 1: Work out A AND B first. Step 2: Then OR with C.

A	B	C	A AND B	(A AND B) OR C
0	0	0	0	0
0	0	1	0	1
0	1	0	0	0
0	1	1	0	1
1	0	0	0	0
1	0	1	0	1
1	1	0	1	1
1	1	1	1	1

Practical applications of logic

Logic gates and truth tables are used in real computing scenarios:

- **Access control:** (hasKey AND knowsCode) OR isAdmin
- **Validation:** (age >= 18) AND (hasConsent)
- **Alarms:** (doorOpen OR windowOpen) AND (alarmOn)

> **Exam tip:** For 3 inputs, there are $2^3 = 8$ rows. Work out intermediate results step by step, using extra columns if needed. Always check you have the right number of rows.

Worked Example 9 — Search and sort algorithms

Question: Complete the table with the name of a search algorithm and a sort algorithm. [2 marks]

Algorithm type	Characteristic	Algorithm name
Search	Is a divide and conquer algorithm	?
Sort	Is NOT a divide and conquer algorithm	?

Model answer:

Search: Binary search **(1)**

Sort: Bubble sort **(1)**

Exam tip: Binary search and merge sort use divide and conquer. Linear search and bubble sort do not.

Worked Example 10 — Algorithm efficiency

Question: Explain one effect the number of comparisons has on the execution time of a sorting algorithm. [2 marks]

Model answer:
A sorting algorithm executes more slowly **(1)**.
Because a large number of comparisons are made **(1)**.
Or: Executes more quickly because a small number of comparisons are made.

Exam tip: Link the cause (more comparisons) to the effect (slower execution) for full marks.

Worked Example 11 — Relational operator results

Question: Here is a relational operator used in a conditional test: count > index. State the two different results of evaluating a conditional test. [2 marks]

Model answer:
True **(1)**
False **(1)**
Note: Do not write 0/1, Yes/No — these were not accepted.

Exam tip: Relational tests always return True or False (Boolean values).

Worked Example 12 — Truth table construction

Question: Complete the truth table for: NOT(A AND B) [4 marks]

Model answer:

A	B	A AND B	NOT(A AND B)
0	0	0	1
0	1	0	1
1	0	0	1

1	1	1	0

Exam tip: Work out intermediate steps first (A AND B), then apply NOT to invert each result.

Worked examples summary

The table below summarises the 12 worked examples covered in this topic, organised by section and difficulty level.

#	Topic	Question type	Marks	Key skill
1	Decomposition	Identify definition	1	Know decomposition = breaking down
2	Subprograms	State benefits	2	Two distinct benefits
3	Abstraction	Explain with example	3	Definition + benefit + example
4	Decomposition	Apply to scenario	2	List specific parts
5	Constructs	Identify in code	3	while vs for...in vs if
6	Syntax error	Define term	1	Breaks language rules
7	Runtime error	Explain	2	Crashes during execution
8	Operators	Calculate result	1	// = integer division
9	Algorithms	Complete table	2	Divide and conquer
10	Efficiency	Explain effect	2	Comparisons affect speed
11	Relational ops	State results	2	True and False
12	Truth tables	Complete table	4	Work out intermediate steps

Quick reference

Key terms and definitions

Term	Definition (mark-scheme style)
Decomposition	Breaking a problem or solution down into smaller, more manageable parts
Abstraction	Hiding or removing unnecessary detail to focus on what is important

Term	Definition (mark-scheme style)
Algorithm	A step-by-step set of instructions to solve a problem or complete a task
Variable	A named storage location whose value can change during program execution
Constant	A named value that does not change during program execution
Syntax error	Code that breaks the rules/grammar of the programming language
Logic error	Code runs but produces incorrect/unexpected results
Runtime error	Code crashes during execution because an operation is impossible
Linear search	Checks each item in turn from the start until found or end reached
Binary search	Divides a sorted list in half repeatedly to find a target (divide and conquer)
Bubble sort	Compares adjacent pairs and swaps if in wrong order; not divide and conquer
Merge sort	Divides list into halves, then merges back in order; divide and conquer

Operators summary

Operator	Name	Example	Result
+	Addition	5 + 3	8
–	Subtraction	5 - 3	2
*	Multiplication	5 * 3	15
/	Division	5 / 2	2.5
//	Integer division	5 // 2	2
%	Modulus	5 % 2	1
**	Exponentiation	5 ** 2	25
==	Equal to	5 == 5	True
!=	Not equal to	5 != 3	True

Programming constructs

Construct	Python pattern	How to identify
Selection	if/elif/else	Look for if keyword and conditions
Condition-controlled loop	while condition:	while keyword with a condition
Count-controlled loop	for i in range():	for with range()
Iteration over structure	for item in list:	for with a list/array name

Typical pitfalls

Pitfall 1 — Confusing decomposition and abstraction.
Wrong: Using the terms interchangeably.
Right: Decomposition = breaking into parts. Abstraction = hiding detail. They are different concepts.

Pitfall 2 — Confusing // and %.
Wrong: Thinking 5 // 2 = 1.
Right: 5 // 2 = 2 (integer division). 5 % 2 = 1 (remainder/modulus).

Pitfall 3 — Confusing while and for...in.
Wrong: Calling both "iteration".
Right: while = condition-controlled loop. for item in list = iteration over data structure.

Pitfall 4 — Wrong relational test results.
Wrong: Saying results are 0/1 or Yes/No.
Right: Relational tests return **True** or **False** (Boolean values).

Pitfall 5 — Giving examples instead of definitions.
Wrong: "Syntax error is like missing a colon."
Right: "Syntax error is code that breaks the rules of the programming language."

Pitfall 6 — Mixing up error types.
Wrong: Saying division by zero is a syntax error.
Right: Division by zero is a **runtime error** — it crashes during execution.

Pitfall 7 — Forgetting binary search needs sorted data.
Wrong: "Binary search can be used on any list."
Right: Binary search only works on **sorted** lists.

Pitfall 8 — Confusing divide and conquer.
Wrong: "Bubble sort uses divide and conquer."
Right: Binary search and merge sort use divide and conquer. Bubble sort and linear search do not.

Pitfall 9 — Truth table rows.
Wrong: Missing rows in a truth table.
Right: 2 inputs = 4 rows. 3 inputs = 8 rows. Use 2^n to calculate.

Pitfall 10 — Vague subprogram benefits.
Wrong: "Makes code easier."
Right: Be specific: "Code only written once and can be called many times" or "Only needs to be debugged in one place."

Assessment Hub | Long-Answer | Notes Index

Williams Physics Education — over **1,100,000+ marks** of exam-aligned content and **4,200+ auto-marked assessments** across 67 qualifications at GCSE, A-Level, AP & Professional Qualifications. Free to use, no login required.

About & Contact • Privacy • Terms • Accessibility • Copyright • DMCA • YouTube

WPE is free for everyone. If our resources help you or your students, you can support the project here.

© 2026 Williams Physics Education. All rights reserved.